Retrieval-Augmented Fine-Tuning for Vietnamese Multimodal Numerical Reasoning

Le Anh Duc **Pham Duc Cong** **Nguyen Huu Thanh**
FPT University, Vietnam
{duclahe200666, congpdhe200419, thanhnhHE204416}@fpt.edu.vn

Abstract

We present Team Arcane's system for the VLSP 2026 shared task on numerical reasoning over Vietnamese multimodal financial reports. Questions draw on numerical evidence in text, tables, charts, or combinations of these modalities, and the task scores the reasoning program behind the answer rather than the final number. A program that reaches the right number from the wrong chart series still scores zero, so a system must name the exact table cell or chart series it reads and apply the same operations as the reference. We adopt retrieval-augmented fine-tuning (RAFT) and fine-tune Qwen3.5-4B with LoRA on prompts containing the report, a chart card listing each chart's labels as read by Qwen3-VL-8B, and ten examples retrieved by BM25. Averaging the weights of three fine-tuned adapters yields 78.66% Program Accuracy and 87.61% Execution Accuracy on the private test set, second on the leaderboard. On held-out reports, fine-tuning raises Program Accuracy from 45.6% to 73.0%, and BM25 examples outperform random ones by 3.7 points.

1 Introduction

The VLSP 2026 shared task on multimodal numerical reasoning asks questions about Vietnamese financial reports and evaluates the program a system writes to compute each answer (VLSP 2026 Organizers, 2026). The evidence is spread across paragraphs, HTML tables, and chart images. The official metric, Program Accuracy (PA), compares the predicted program against the reference, which frames the task as semantic parsing. A correct program must preserve the reference's reasoning structure and select the corresponding evidence from the report. Chart values exist only within images, so a program must name them by series and category, and a correct number read from the wrong chart series receives a zero.

We adopt the retrieval-augmented fine-tuning (RAFT) strategy (Zhang et al., 2024), which fine-tunes a model on prompts containing retrieved context. It suits this task because programs recur across reports. For example, a question about how far an index moved between two dates needs the same operations for every company, and only the chart and its labels change. Solved questions from other reports can therefore suggest the program's structure, meaning the program with its source names, labels and copied numbers replaced by placeholders. Two programs share a structure when they apply the same operations in the same order to the same earlier steps and constants such as 100. We fine-tune Qwen3.5-4B (Qwen Team, 2026) with LoRA (Hu et al., 2022), the *writer*, to translate a prompt into a program (Figure 1). Each prompt includes the report, the question, a *chart card* for each chart with the title, series and categories that Qwen3-VL-8B reads from the image (Bai et al., 2025), and ten examples that BM25 (Robertson and Zaragoza, 2009) retrieves from other reports. Following prior work on in-context learning (Liu et al., 2022; Rubin et al., 2022) and retrieval-augmented semantic parsing (Pasupat et al., 2021; Gupta et al., 2022), we put retrieved examples in both training and inference prompts. At inference, we average the weights of three fine-tuned runs into a single adapter, following model soups (Wortsman et al., 2022).

Our system achieves 78.66 PA on the private test set. On held-out reports, fine-tuning on prompts with retrieved examples raises PA from 45.6 to 73.0, while the writer trained with ten examples produces almost no valid programs once they are removed. Ten BM25 examples outperform random ones by 3.7 points, and at least one of them shares the structure of the reference program for 70% of questions, against 32% for random examples. For three 2-epoch runs, averaging

their weights matches an ensemble that rescores the three runs' programs (77.45 vs. 77.28 private PA) with one decoding pass instead of three. Finally, we analyze some of the remaining errors with samples from the public test set.

2 Related Work

Numerical Reasoning over Financial Reports. FinQA introduced executable programs over the text and tables of financial reports (Chen et al., 2021), and TAT-QA extended this to questions that combine table cells with text (Zhu et al., 2021). The VLSP 2025 edition brought the task to Vietnamese text and tables (Le et al., 2025). Its top systems fine-tuned 7B and 8B writers and added GRPO (Chu et al., 2025; Nguyen et al., 2025a), or voted over programs from several reasoning paths (Nguyen et al., 2025b). The 2026 data add chart images and dedicated chart operations, so scores are not comparable across years, and the writer must now also locate values that appear only in images.

Multimodal Reasoning with Charts. ChartQA introduced questions whose answers require reading values from chart images (Masry et al., 2022), and DePlot translates a chart into a table that a language model then reasons over (Liu et al., 2023). In this task, a program reads chart values through its own operations, so the writer needs to know which series and categories exist rather than their values. Our chart cards therefore keep only the title, series, and categories.

Retrieval-Augmented Semantic Parsing. Retrieving examples similar to the input improves in-context learning over random ones (Liu et al., 2022), and retrievers trained for the task improve semantic parsing further (Rubin et al., 2022). For compositional outputs, examples help most when together they cover the structure of the target (Levy et al., 2023). Other work moves the examples into training. MetaICL fine-tunes a model on prompts with examples (Min et al., 2022), CASPER and RetroNLU fine-tune semantic parsers on retrieved input–parse pairs and retrieve again at inference (Pasupat et al., 2021; Gupta et al., 2022), and RAFT fine-tunes on prompts with retrieved documents (Zhang et al., 2024). We follow this line, with training questions and their reference programs as the retrieved examples.

Split	Reports	Questions	4+ steps	Chart
Training	79	1,725	12	39
Public test	19	459	19	44
Private test	41	980	–	–

Table 1: Data splits. The last two columns give the share (%) of reference programs with 4+ steps and with a chart read. Private programs are hidden.

Combining Fine-Tuned Runs. Model soups average the weights of models fine-tuned from the same initialization, approaching ensemble accuracy at the inference cost of a single model (Wortsman et al., 2022). Ensembles instead combine the models at decoding time, for instance by re-scoring each model's outputs under all of them (Sennrich et al., 2016). We compare both on LoRA runs of the same writer.

3 Task and data

Input and output. Each question comes with one report, made of paragraphs, tables (`Table 1`, ...) and charts (`Image 1`, ...). Tables arrive as HTML and charts as PNG images. A system outputs a program, a list of steps separated by semicolons. Each step is one of two kinds.

- A *computation step* applies `add`, `subtract`, `multiply`, `divide`, `exp` or `greater` to two arguments. Each argument is a number copied from the report, a constant such as 100, or the result `#0`, `#1`, ... of an earlier step.
- A *read step* takes the maximum, minimum, sum or average of table cells (`table_max`, ...) or chart values (`chart_max`, ...), or reads one chart value (`chart_at`) or a category total (`chart_total`). It names the source and up to three labels, such as a series and a category or the two ends of a range, with `none` for an unused slot.

Example. The question of Figure 1 asks how many percentage points VN-Index moved between two dates on the report's first chart. Its program reads both values with `chart_at` and subtracts them (Figure 1, bottom). Another question on the same chart asks for the average of VN-Index over the four dates and takes it in one step, `chart_average(Image 1; VN-Index; Apr-20; Jan-21)`, with the range given by its first and last category.

Splits. The organizers provide training, public-test and private-test splits with disjoint sets of re-

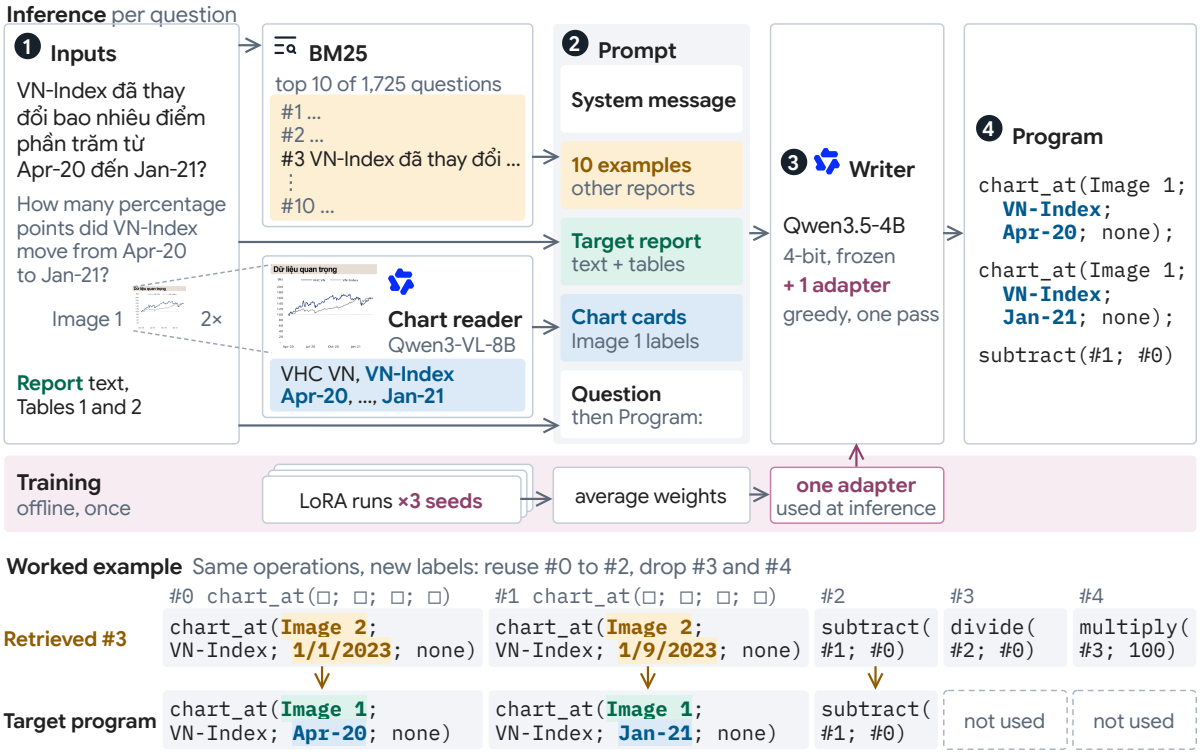


Figure 1: System overview. Inference: BM25 examples and the card of the enlarged chart fill one prompt, and the writer outputs the program. Training: three LoRA runs are averaged into the one adapter the writer uses. Worked example: the target reuses steps #0 to #2 of retrieved example 3 with new labels and drops #3 and #4.

ports (Table 1). Public programs are longer. Of the public reference programs, 19% have four or more steps, against 12% of training ones, and the mean length rises from 2.3 to 2.7 steps. The writer therefore sees long programs less often in training than it meets them on the public test.

Metrics. PA counts a question correct when the predicted program matches the reference. Execution Accuracy (EA) counts it correct when the predicted program computes the reference answer. EA is the looser metric, since several programs can compute the same value.

4 System

The system builds one prompt per question and fine-tunes a writer to complete it with a program (Figure 1). Each prompt holds examples retrieved from other reports, the target report with a card for each chart, and the question.

Retrieval. Retrieval looks for solved questions whose programs the target may share. BM25 (Robertson and Zaragoza, 2009) with $k_1 = 1.5$ and $b = 0.75$ ranks the training questions against the target question by their case-folded word tokens and keeps the top ten, each as a question with its program. Questions from the target's own report are skipped, because they would give away its labels and numbers. Test prompts retrieve from all 1,725 training questions, and training prompts from the other reports.

Chart cards. A program selects a chart value by naming its series and category, as in `chart_at(Image 1; VN-Index; Apr-20; none)`, so the writer needs each chart's labels but not its values. The chart reader, Qwen3-VL-8B-Instruct (Bai et al., 2025), writes these labels into the chart card, a JSON object with the title, series labels and category labels (Figure 1). It is asked to copy visible text exactly, including Vietnamese diacritics and date formats, and to skip axis tick values. Each chart is enlarged to twice its stored size with Lanczos resampling and read once with greedy decoding, so training and test share the same cards.

Prompt. The system message states the task and the program grammar. The user message lists the ten examples, the full target report (paragraphs, tables as HTML, and each chart's name followed by

its card) and the question (Appendix A).

Writer. The writer is Qwen3.5-4B, frozen in 4-bit NF4 (Dettmers et al., 2023), with LoRA adapters (Hu et al., 2022) of rank 32 ($\alpha = 64$, no dropout) on the attention and MLP layers. It trains for three epochs with the loss on program tokens only, using 8-bit AdamW, learning rate 2×10^{-4}, cosine decay after 5 warm-up steps, weight decay 0.001 and an effective batch of 8. Training prompts longer than 8,192 tokens are dropped. At test time the writer decodes greedily, with thinking disabled and at most 512 new tokens. We train three runs that share their LoRA initialization and differ only in their data-order seed.

Combining runs. The *adapter soup* averages the weight updates of the three runs, as uniform model soups average whole models (Wortsman et al., 2022):

$$\bar{\theta} = \theta_0 + \tfrac{1}{3} \textstyle\sum_{s=1}^{3} \tfrac{\alpha}{r} B_s A_s. \qquad (1)$$

Stacking the three runs' factors gives one adapter of rank 96 with $\alpha = 96$, so its scale α/r is 1, and each A_s is scaled by $\frac{1}{3} \cdot \frac{64}{32} = \frac{2}{3}$. This adapter stores the mean exactly, which we confirmed numerically, and decodes at the cost of a single run. The *sequence ensemble* instead lets each run k write a program y_k greedily and returns the one with the highest summed log-likelihood under all three runs (Sennrich et al., 2016):

$$\hat{y} = \arg\max_{k} \textstyle\sum_{s=1}^{3} \log p_s(y_k \mid x). \qquad (2)$$

It skips scoring when the runs agree and otherwise needs up to nine scoring passes on top of the three generation passes.

5 Experimental setup

Implementation. The submitted system uses only the official training data, with the chart cards as its only model-generated input[1], and calls no external API. The writer (`Qwen/Qwen3.5-4B`) and the chart reader (`Qwen/Qwen3-VL-8B-Instruct`) are within the task's 8B limit by the nominal sizes of their official releases. Writers are trained with Unsloth and TRL, and chart cards are generated with vLLM, on single A100 40 GB GPUs on Google Colab.

[1]The chart cards of all splits and the script that generated them are at `https://huggingface.co/datasets/lducc/vlsp2026-chart-cards`.

Combination	Epochs	Passes	PA	EA
Adapter soup*	3	1	**78.66**	87.61
Adapter soup	2	1	77.45	**88.47**
Sequence ensemble	2	3	77.28	88.30

Table 2: Official private test results. *Submitted. Passes are writer generation passes per question.

Team	Rank	PA	EA
thanhducsvbg	1	**79.17**	**89.33**
Arcane (ours)	2	<u>78.66</u>	<u>87.61</u>
Taekz	3	76.42	87.44
bachanglinhngulam	4	74.35	86.92
rudeus	5	73.32	85.71
HUNTER	6	72.46	83.30

Table 3: Official private leaderboard, top six teams. Best in bold, second underlined.

Scoring. All test scores in Tables 2 and 3 come from the official private leaderboard. Public scores used for the submission choice and the error analysis come from our own PA checker, a randomized equivalence test. It treats the reference's labels and numbers as symbols and compares the two expressions at three random assignments, so `add(a; b)` matches `add(b; a)`. On the submissions we checked, its public totals equal the official ones, which does not guarantee agreement on every program. A program is valid when it parses, uses known operations with the right number of arguments, refers only to earlier steps and names a table or chart that exists in the report.

Development folds. We split the training set by report into five folds of 355, 340, 333, 360 and 337 questions. A fold is scored by a writer trained on the other four. Most ablations use folds 0 to 2 (1,028 questions, 47 reports), and the tables mark those that pool all five folds or use fold 0 alone. To save compute, these development writers train for one epoch with LoRA rank 16 ($\alpha = 32$, dropout 0.05) and Qwen3-VL-4B cards. Each ablation changes one factor on the same questions.

6 Results

Leaderboard. The submitted 3-epoch adapter soup ranks second by PA on the private leaderboard (Table 3). It trails the first team by 0.51 points.

Soup or ensemble. Averaging the adapter weights reaches similar PA with one writer generation pass rather than three. On the 2-epoch runs,

Question (English translation)	Reference program	Submitted program
Same quantity, different operations		
Average change (%) of VN30, VNMidcap, and VNSmallcap in 9/2022?	`table_average(Table 1; T9/2022; VN30; VNSmallcap)`	`add(-11.5; -12.7); add(#0; -14.3); divide(#1; 3)`
Wrong source		
Total foreign buying value over the October sessions?	`chart_sum(Image 6; NĐTNN Mua; none; none)`	`table_sum(Table 1; T10/2022; none; none)`
Missing step		
Ratio of the tracked banks' 3Q24 net interest income to VCB's market cap?	`chart_at(Image 1; NII; 3Q24 (TTM); none); multiply(#0; 1000); divide(#1; 495752)`	`chart_at(Image 1; NII; 3Q24 (TTM); none); divide(#0; 495752)`

Table 4: Public test errors of the submitted system.

		Examples at test time		
Writer	Trained with	None	Random	BM25
Base model	–	–	–	45.6
Fine-tuned	3 BM25	42.7	67.4	72.5
	10 BM25	0.3	69.3	**73.0**

Table 5: Held-out PA on folds 0 to 2 (1,028 questions). Fine-tuned writers are tested with as many examples as in training, the base model with ten.

Example set	Same structure (%)	PA
Retrieved		
Random	32.1	69.3
BGE-M3 dense	68.6	73.6
BM25	70.0	73.0
Oracle, chosen by the reference program		
BM25, same-structure removed	0.0	70.5
Same-structure first, random rest	92.5	77.8
Same-structure first, BM25 rest	92.5	79.0

Table 6: Held-out PA with ten examples on folds 0 to 2, for the writer trained with ten BM25 examples. Same structure is the share of questions with an example of the reference program's structure (Section 1).

the soup scores 77.45 private PA and the sequence ensemble 77.28 (Table 2). The ensemble may also need passes to score candidate programs when the runs disagree.

Epochs. Three epochs raise private PA over two by 1.2 points and lower private EA by 0.9. On development fold 1, three epochs beat two by 15 questions with one data-order seed and lost by 9 with another, so these differences describe one submission rather than a tested effect.

7 Ablations

The ablations ask what fine-tuning adds, what examples add and which examples help. They use the development writers of Section 5 on held-out folds.

Fine-tuning adds most of the accuracy. With the same ten BM25 examples in the prompt, fine-tuning raises held-out PA from 45.6 to 73.0 (Table 5). The base model already writes valid programs for 87.5% of questions, so most of the gain comes from choosing the right steps and labels. Every writer was trained with examples, so this gain does not separate fine-tuning on examples from plain fine-tuning.

Examples keep the format. Without examples, the writer trained with ten produces 5 valid programs out of 1,028 (Table 5). It separates arguments with commas or writes steps in words, such as `subtract #0 from #1`. The writer trained with three keeps more of the format and scores 42.7. Random examples restore the format, which suggests that the writers learned it from the examples more than from the grammar in the instruction.

Examples with the same structure help most. BM25 examples beat random ones by 3.7 points with ten examples and 5.1 with three (Table 5). BM25 also more often retrieves an example with the reference program's structure, for 70.0% of questions against 32.1% (Table 6). Putting same-structure examples first raises PA to 79.0, and removing them from the BM25 list lowers it to 70.5. These oracle sets choose examples by the reference program, so they are diagnostic rather than a usable retriever. They are consistent with a benefit from including a same-structure example, al-

though they do not establish that structure alone explains the gain. With same-structure examples first, BM25 examples for the rest add 1.2 points over random ones. BGE-M3 (Chen et al., 2024) dense examples have the same structure slightly less often than BM25 (68.6 against 70.0%) and score 0.6 points higher. All runs use one seed.

Other choices. Appendix B compares writer and chart reader models and two inputs we dropped. Qwen3.5-4B was the strongest writer at or below 8B among those we tried, and swapping in Qwen3-VL-8B cards at test time beat Qwen3-VL-4B cards by 1.2 points.

8 Error analysis

Table 4 shows three public test programs of the submitted system that use different operations or sources from the reference, so they are wrong under any reading of PA. In the first, the system adds the three values by hand where the reference reads their average with `table_average`. Both compute the same number, which EA counts as correct and PA does not. In the second, the system reads a table where the reference reads a chart. In the third, it drops the unit conversion `multiply(#0; 1000)`.

9 Conclusion

We fine-tuned a 4B writer on prompts with ten BM25 examples and chart cards and averaged three runs into one adapter. It reached 78.66 private PA, second in VLSP 2026, with one writer decoding pass per question after the chart cards are generated. On held-out reports, oracle ablations suggest that including an example with the reference program's structure can improve PA, and BM25 finds none for 30% of questions. Retrieving examples by predicted structure may help these questions, which we have not tested.

Limitations

Ablations use single seeds and a cheaper development writer than the submitted one. The public test file released to participants (`public_test.json`) contains a reference program for each of its 459 questions, although the task page describes evaluation references as retained by the organizers (VLSP 2026 Organizers, 2026). We used these programs for our checker's public scores and the error analysis, never for training, and public PA informed the final choice between systems. The system is tuned to one annotation style and one program language, and we have not tested it on other reports or tasks.

References

Shuai Bai, Yuxuan Cai, Ruizhe Chen, et al. 2025. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*.

Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.

Duc Dinh Chu, Thanh-Bac Nguyen Ba, Duy Dinh Le, and Khanh Van Tran. 2025. Enhancing numerical reasoning in vietnamese financial question answering through program-centric policy optimization. In *Proceedings of the 11th International Workshop on Vietnamese Language and Speech Processing*, pages 197–204.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*.

Vivek Gupta, Akshat Shrivastava, Adithya Sagar, Armen Aghajanyan, and Denis Savenkov. 2022. RetroNLU: Retrieval augmented task-oriented semantic parsing. In *Proceedings of the 4th Workshop on NLP for Conversational AI*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Ngoc Toan Le, My Linh Ha, Thi Duc Pham, The Quyen Ngo, and Thi Minh Huyen Nguyen. 2025. VLSP 2025 challenge: Numerical reasoning question and answer. In *Proceedings of the 11th International Workshop on Vietnamese Language and Speech Processing*, pages 187–196.

Itay Levy, Ben Bogin, and Jonathan Berant. 2023. Diverse demonstrations improve in-context compositional generalization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.

Fangyu Liu, Julian Eisenschlos, Francesco Piccinno, Syrine Krichene, Chenxi Pang, Kenton Lee, Mandar Joshi, Wenhu Chen, Nigel Collier, and Yasemin Altun. 2023. DePlot: One-shot visual language reasoning by plot-to-table translation. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 10381–10399.

Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022. What makes good in-context examples for GPT-3? In *Proceedings of Deep Learning Inside Out (DeeLIO 2022)*.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. 2022. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2263–2279.

Sewon Min, Mike Lewis, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2022. MetaICL: Learning to learn in context. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.

Khac Duy Nguyen, Quang Tri Vo, Ho Bao Nhat Le, Cong Huy Hoang Dinh, Quang Huy Ngo, and Huy Tien Nguyen. 2025a. MoFin: A small vietnamese language model for financial reasoning via reinforcement learning. In *Proceedings of the 11th International Workshop on Vietnamese Language and Speech Processing*, pages 205–213.

Quang Nguyen, Vu Thanh Dat Ha, Thanh Tuan Le, and Huan Vu. 2025b. A graph-based agent approach to numerical reasoning question answering. In *Proceedings of the 11th International Workshop on Vietnamese Language and Speech Processing*, pages 223–239.

Panupong Pasupat, Yuan Zhang, and Kelvin Guu. 2021. Controllable semantic parsing via retrieval augmentation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.

Qwen Team. 2026. Qwen3.5: Towards native multimodal agents.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Ohad Rubin, Jonathan Herzig, and Jonathan Berant. 2022. Learning to retrieve prompts for in-context learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Edinburgh neural machine translation systems for WMT 16. In *Proceedings of the First Conference on Machine Translation: Volume 2, Shared Task Papers*, pages 371–376.

VLSP 2026 Organizers. 2026. Numerical Reasoning QA: Shared-task description and evaluation protocol. Accessed October 2026.

Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. 2022. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proceedings of the 39th International Conference on Machine Learning*.

An Yang, Anfeng Li, Baosong Yang, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Tianjun Zhang, Shishir G. Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E. Gonzalez. 2024. RAFT: Adapting language model to domain specific RAG. In *First Conference on Language Modeling*.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*.

A Prompt

The system message is the same for every question:

System message

```
Write an exact reasoning program for the
TARGET Vietnamese financial question. Return
only the program, without the computed answer,
explanations, tags, or markdown.
EXAMPLE question/program pairs illustrate the
DSL; their source IDs and values belong to
other documents. Ground the TARGET program in
its own text, tables, and chart labels.

Allowed operations:
- Arithmetic, exactly 2 arguments: add,
subtract, multiply, divide, exp, greater
- Table reads, exactly 4 arguments: table_max,
table_min, table_sum, table_average
- Chart reads, exactly 4 arguments: chart_at,
chart_max, chart_min, chart_sum,
chart_average, chart_total

Write each step as operation(arg1; arg2; ...).
Separate steps with semicolons. #0 refers to
the first step's result, #1 to the second, and
so on. Reference only earlier steps. The first
argument of a table or chart read is its exact
Table k or Image k source name. Copy selector
labels and evidence numeric literals exactly,
```

```
preserving spelling, punctuation, and
formatting. Arithmetic constants such as 1 or
100 are allowed when required by the question.
Use none for unused selector arguments.
chart_total(Image k; primary_name; none; none)
gives the total at a primary chart category
(e.g. Jan-25).
```

The user message for the question of Figure 1 follows. The first and third of the ten examples are shown. Grey notes explain each part and are not in the prompt, and … marks cut text.

User message (abridged)

Ten retrieved examples, most similar first. Each is a question from another report and its reference program.

```
EXAMPLE
Question: Trong giai đoạn từ Jun 19 đến
Jun 21, nếu lấy phần trăm thay đổi của NKG
VN trừ ...
Program: chart_at(Image 1; NKG VN; Jun 19;
none); chart_at(Image 1; NKG VN; Jun 21;
none); . . .
```

(example 2)

```
EXAMPLE
Question: VN-Index đã thay đổi bao nhiêu
phần trăm từ 1/1/2023 đến 1/9/2023?
Program: chart_at(Image 2; VN-Index; 1/1/2023;
none); chart_at(Image 2; VN-Index; 1/9/2023;
none); subtract(#1; #0); divide(#2; #0);
multiply(#3; 100)
```

(examples 4 to 10)

The target report: its paragraphs, then each table as HTML.

```
TARGET
TEXT
CTCP Tập Đoàn Vĩnh Hoàn (HOSE: VHC) Lạc
quan hậu Covid-19
```

(six more paragraphs)

```
Table 1
<table><tr><td>(%)</td><td>1 tháng</td>
<td>6 tháng</td><td>12 tháng</td></tr><tr>
<td>Tuyệt đối</td><td>-6.3</td>...</table>
```

(Table 2)

Each chart: its name and file, then its chart card. The writer never sees the image.

```
IMAGES
Image 1:
97d35f52-8e2f-4484-a55e-652c76178226.png
{"title": "Dữ liệu quan trọng",
"series_labels": ["VHC VN", "VN-Index"],
"x_axis_labels": ["Apr-20", "Jul-20",
"Oct-20", "Jan-21"]}
```

The target question. The writer continues after Program:.

```
Question: VN-Index đã thay đổi bao nhiêu
điểm phần trăm từ Apr-20 đến Jan-21?
Program:
```

B Other design choices

Writer. Qwen3.5-4B was the strongest writer at or below 8B that we compared (Table 7). It beat the larger Qwen3-8B (Yang et al., 2025) by 4.1 points, so we spent the compute on more runs of it.

Component	Model	PA
Writer	**Qwen3.5-4B**	**72.5**
	Qwen3-8B	68.4
	Qwen3-4B-2507	67.4
Chart reader	**Qwen3-VL-8B**	**74.0**
	Qwen3-VL-4B	72.8

Table 7: Held-out PA with other models, submitted choice in bold. Writers use folds 0 to 2 and three examples, chart readers all five folds and ten.

Chart reader. The writers in Table 7 were trained with Qwen3-VL-4B cards, and swapping in Qwen3-VL-8B cards at test time fixed 25 programs and broke 5.

Chart images. Adding the chart images to the prompt next to the cards lowered PA from 73.8 to 71.3 on fold 0.

Selected report units. Replacing the full report at test time with the five paragraphs, table rows or cards that BGE-M3 (Chen et al., 2024) and a fine-tuned reranker rank highest lowered public PA from 66.2 to 63.8 for a writer trained on full reports, in our one comparison on the public test.